

Tutorial - Semana 4, Encontro 1

Introdução

A Semana 3 encerrou o Módulo 1 com o primeiro build executável do Vertical Slice: um nível de teste com Player controlável, terreno, material e iluminação global. Nada desse trabalho é refeito agora. Este encontro abre o Módulo 2 (Construir Sistemas) e desloca o foco de composição visual para arquitetura de gameplay: a primeira pergunta deixa de ser "como o mundo aparece" e passa a ser "onde vive o estado que uma partida precisa compartilhar entre cenas". A resposta do Godot para esse problema é o **Autoload/Singleton**, e o primeiro sistema construído sobre ele é o `GameManager`.

Neste encontro o `GameManager` é construído como uma **Orchestration** (visual scripting do Orchestrator), a camada de scripting principal da disciplina. Cada trecho de GDScript apresentado no tutorial é a **implementação de referência** da lógica — o que o grafo do Orchestrator deve fazer, nó a nó —, não um arquivo a ser digitado.

Objetivos da semana

- Explicar Autoload/Singleton como mecanismo nativo do Godot para estado global compartilhado entre cenas.
- Diferenciar o papel do `GameManager` (regras de partida e estado compartilhado) de um Node comum de cena.
- Criar um `GameManager` como Orchestration e registrá-lo como Autoload no projeto do Vertical Slice.
- Dar ao `GameManager` sua primeira responsabilidade concreta: `spawn_player()`, que posiciona o Player no `PlayerStart` (Marker3D no nível) ao carregar — o equivalente ao `ChoosePlayerStart` da Unreal.

Resultado esperado ao final da semana

Ao final da Semana 4 (Encontros 1 e 2), cada estudante terá, além do projeto herdado das Semanas 1–3, um `GameManager` e um `SaveManager` configurados como Autoload, com pelo menos um dado de progresso persistindo entre cenas. Este tutorial cobre apenas o **Encontro 1**: a criação da Orchestration `GameManager`, seu registro como Autoload e a implementação de `spawn_player()`, que posiciona o Player no `PlayerStart` do nível.

Pré-requisitos

- Projeto do Vertical Slice com o nível de teste, Player e build da Semana 3 (ver Tutorial - Semana 3, Encontro 2).
 - Uso básico do editor do Orchestrator (criar grafo, adicionar nós, ligar pinos de execução e de dados), praticado no Módulo 1.
 - Noções básicas de GDScript (variáveis, funções) para ler as implementações de referência, cobertas informalmente nas Semanas 1–3.
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto do Vertical Slice completo até a Semana 3, com `level_exploration.tscn`, Player controlável e build exportado.

Arquivos necessários

- Nenhum arquivo externo. O `GameManager` é uma Orchestration nova, criada dentro do próprio projeto.

Assets utilizados

- Nenhum asset novo. O encontro é de arquitetura de código/Orchestrator; a única alteração de cena do nível é um `Marker3D` (`PlayerStart`) adicionado na Parte 3.

Projeto esperado

- Projeto aberto no Godot 4.7 com o Orchestrator instalado e ativado, e a Scene `level_exploration.tscn` do Encontro 2 da Semana 3 já funcionando.
- Pasta `orchestrations/autoload/` criada (ou pronta para ser criada), conforme a estrutura definida no `PROJECT_ARCHITECTURE.md` (seção 8).

Imagem sugerida

Objetivo: mostrar a estrutura de pastas recomendada do projeto no FileSystem Dock, com a pasta `orchestrations/autoload/` em destaque. Enquadramento: captura de tela do FileSystem Dock do Godot, árvore de pastas expandida. Elementos importantes: pastas `scenes/`, `orchestrations/`, `orchestrations/autoload/`. Destaque: a pasta `orchestrations/autoload/`, onde o `game_manager.torch` será criado. Legenda sugerida: "Estrutura de pastas do projeto, com orchestrations/autoload/ pronta para receber o GameManager."

Parte 1 — Autoload/Singleton: onde vive o estado global

Objetivo

Entender por que nenhuma Scene individual é o lugar certo para guardar dados que várias cenas precisam compartilhar, antes de criar qualquer grafo.

Conceito

Toda engine de jogos que trabalha com múltiplas cenas precisa resolver o mesmo problema: onde vive um dado ou uma regra que não pertence a nenhuma cena específica, mas que todas precisam acessar — pontuação da partida, condição de vitória, referência ao jogador ativo. Se esse dado ficar dentro de uma Scene comum, ele desaparece assim que a Scene é trocada, porque o Godot descarrega a árvore de nós anterior ao carregar a próxima.

O Godot resolve isso de forma nativa e formal com **Autoload/Singleton**: um recurso registrado nas configurações do projeto é instanciado automaticamente na raiz da árvore de cenas assim que o jogo inicia, e permanece acessível globalmente — por nome, de qualquer script ou Orchestration do projeto — sobrevivendo a qualquer troca de cena. Esse recurso pode ser um script GDScript (`.gd`), uma Orchestration (`.torch`) ou uma cena (`.tscn`): o Orchestrator suporta selecionar uma Orchestration diretamente no campo de registro do Autoload, exatamente como um `.gd`. O `GameManager` construído neste encontro ocupa esse papel, reunindo em um único ponto o que a Unreal separa formalmente em GameMode (regras) e GameState (estado compartilhado), sem que o Godot precise de dois conceitos distintos para isso.

Passo a passo

1. Reabra o projeto do Vertical Slice e confirme que a Scene `level_exploration.tscn` da Semana 3 abre e roda normalmente (F6).
2. No FileSystem Dock, crie a subpasta `orchestrations/autoload/`, caso ainda não exista.
3. Clique com o botão direito sobre `orchestrations/autoload/` e crie uma nova Orchestration chamada `game_manager.torch` (o nome exato do item de menu do Orchestrator pode variar conforme a versão — consultar a documentação oficial).
4. Abra a Orchestration recém-criada no editor do Orchestrator. Na aba de configuração do grafo, confirme (ou defina) que a classe base é `Node` — um Autoload de gerência não precisa de nada mais específico.
5. Registre uma nota curta no grafo descrevendo a responsabilidade do `GameManager`: regras de partida e estado compartilhado. Isso cumpre o mesmo papel do comentário de topo de um script (`PROJECT_ARCHITECTURE.md`, seção 10).
6. Salve a Orchestration (**Ctrl+S**).

Resultado esperado

Existe um arquivo `orchestrations/autoload/game_manager.torch` com a nota de responsabilidade registrada, ainda sem variáveis nem funções — o conteúdo é construído na Parte 2 (registro como Autoload) e na Parte 3 (`spawn_player()`).

Verificando

1. Confirme que o arquivo aparece em `orchestrations/autoload/game_manager.torch` no FileSystem Dock.
2. Abra a Orchestration e confirme que a classe base é `Node` e que não há erros no editor.

Problemas comuns

- Criar a Orchestration como um grafo anexado a um Node de uma Scene do nível (como `NívelTeste`) em vez de um arquivo solto em `orchestrations/autoload/`: um Autoload não é um Node dentro da cena do nível — ele é registrado separadamente em Project Settings, como será feito na Parte 2.
- Definir uma classe base diferente de `Node` (por exemplo, `Node3D`): um Autoload de gerência não vive no espaço 3D e não precisa de transform.

Boas práticas

- Manter toda Orchestration de Autoload em `orchestrations/autoload/`, nunca solta na raiz de `res://`, conforme a estrutura de pastas do `PROJECT_ARCHITECTURE.md`.
- Registrar, desde o primeiro grafo, uma nota curta explicando a responsabilidade do Autoload — isso evita que, nas semanas seguintes, o `GameManager` acumule responsabilidades que deveriam pertencer ao `SaveManager` ou a um `Component`.

Comparação com Unity

A Unity não possui um mecanismo formal equivalente ao Autoload — o mesmo problema de estado global é resolvido por convenção do próprio time, tipicamente com `DontDestroyOnLoad` aplicado a um `GameObject`, ou com um singleton implementado manualmente em C# (por vezes apoiado em um `ScriptableObject`). O Godot formaliza esse padrão como um recurso de primeira classe do editor, registrado explicitamente em Project Settings, o que reduz a chance de implementações inconsistentes entre membros de uma mesma equipe — e permite que a lógica desse singleton seja escrita em GDScript ou em um grafo visual, sem mudar o mecanismo.

Parte 2 — Registrando o GameManager como Autoload

Objetivo

Transformar a Orchestration criada na Parte 1 em um Singleton acessível globalmente, e validar esse acesso a partir de outra cena.

Conceito

Criar a Orchestration não é suficiente para torná-la um Autoload: o Godot só instancia automaticamente na inicialização do jogo os recursos explicitamente registrados na aba **Autoload** de Project Settings. Esse registro associa um nome global (por convenção, igual ao nome desejado do Singleton) ao caminho do recurso, e é esse nome que passa a funcionar como um ponto de acesso único de qualquer lugar do projeto — de um script GDScript ou de outra Orchestration — sem precisar instanciar o Node manualmente ou passar referências entre cenas.

Passo a passo

1. Abra **Project > Project Settings** e selecione a aba **Autoload**.
2. No campo **Path**, clique no ícone de pasta e selecione `res://orchestrations/autoload/game_manager.torch`.
3. No campo **Node Name**, confirme que aparece `GameManager` (ajuste manualmente para PascalCase, se necessário).
4. Clique em **Add** para registrar o Autoload.
5. Confirme que `GameManager` aparece na lista de Autoloads, com a caixa **Enable** marcada.
6. Feche Project Settings e abra a lógica do Player (`player.tscn` e sua Orchestration ou script, da Semana 1).
7. Em uma parte já existente da lógica do Player (por exemplo, no evento `_ready`), adicione temporariamente uma chamada que imprima o `GameManager`, para testar o acesso ao Autoload por nome, sem `get_node` nem referência:

```
func _ready() -> void:
    print(GameManager) # teste temporário: resolve o Autoload
                        por nome
```

No Orchestrator, isso é um nó de evento `_ready` ligado a um nó **Print** que recebe, como entrada, um nó **Get Autoload** apontando para `GameManager`.

8. Rode a Scene `level_exploration.tscn` com F6 e observe, no painel **Output**, que o `GameManager` foi impresso corretamente — confirmando que ele existe e é acessível a partir do Player.
9. Remova o teste (`print` /nó Print), já que ele não faz parte da funcionalidade final.
10. Salve o Player e o projeto (**Ctrl+S**).

Resultado esperado

O `GameManager` aparece na lista de Autoloads em Project Settings, habilitado, e é acessível por nome (`GameManager`) a partir de qualquer script ou Orchestration do projeto — validado a partir de um teste rápido no Player.

Verificando

1. Reabra **Project > Project Settings > Autoload** e confirme que `GameManager` está listado e habilitado.
2. Rode a Scene e confirme, no Output, que o teste de acesso a partir do Player não gera erro (`Invalid get index` ou `Identifier not declared`).

3. Repita o teste de acesso a partir de um Node diferente do Player, confirmando que o Autoload não depende de qual cena está ativa.

Problemas comuns

- Registrar o Autoload com o **Node Name** em minúsculo ou diferente de `GameManager` : isso obriga a usar um nome inconsistente em todo o projeto — sempre ajustar manualmente o campo Node Name para PascalCase antes de clicar em Add.
- Confundir Autoload com um Node comum adicionado manualmente à Scene do nível: um Node adicionado dentro de `level_exploration.tscn` desaparece ao trocar de cena; o Autoload vive fora da árvore de cenas do nível e só é configurado em Project Settings.
- Testar o acesso ao `GameManager` antes de salvar o registro em Project Settings, recebendo um erro de identificador não declarado: sempre confirmar que o Autoload está habilitado antes de rodar a Scene de teste.

Boas práticas

- Sempre testar o acesso ao Autoload a partir de uma cena diferente da que você está editando, antes de considerar o registro concluído — é exatamente esse comportamento (sobreviver à troca de cena) que justifica o uso de Autoload em vez de uma variável comum.
- Remover qualquer teste (`print` /nó Print) antes de seguir para a próxima etapa, mantendo o grafo limpo.
- Nunca duplicar em uma Scene um dado que já pertence ao `GameManager` — se o dado precisa ser compartilhado entre cenas, ele pertence ao Autoload, não a uma variável local de um Node específico.

Comparação com Unity

Na Unity, o equivalente a esse registro seria implementar manualmente o padrão singleton em C# — geralmente uma propriedade estática que retorna a única instância existente, combinada com `DontDestroyOnLoad` para que o GameObject sobreviva à troca de cena. O Godot elimina essa etapa de implementação manual: o registro em Project Settings já garante instância única e sobrevivência entre cenas, sem exigir nenhum código (ou nó) de controle de instância no próprio grafo.

Parte 3 — PlayerStart: o GameManager decide onde o Player nasce

Objetivo

Dar ao `GameManager` sua primeira responsabilidade concreta: posicionar o Player, ao carregar o nível, na posição de um Node marcador da cena — sem que a coordenada de spawn viva no próprio `GameManager`.

Conceito

Toda engine precisa de um ponto que decida onde o jogador aparece ao entrar em um nível. A Unreal formaliza isso em dois pedaços: um actor `PlayerStart` colocado no mapa (a coordenada) e `GameMode.ChoosePlayerStart` (a decisão de qual usar). O Godot não formaliza nenhum dos dois — cabe ao projeto montar o padrão: um `Marker3D` no nível guarda a coordenada, e o `GameManager` consulta esse marcador ao carregar. A separação é a mesma lição do Autoload: a **coordenada** é dado da cena (muda quando o level designer arrasta o marcador); a **política** ("nascer no início do nível") é responsabilidade de um gerenciador. Nunca o contrário — uma posição fixa escrita dentro do `GameManager` obrigaria a editar o grafo para mover o ponto de partida.

Neste encontro, `spawn_player()` só conhece o `PlayerStart`. Na Semana 7, quando o `Checkpoint` existir, a mesma função passará a escolher entre o `PlayerStart` e o último checkpoint alcançado.

Passo a passo

1. Abra `level_exploration.tscn`. Adicione um Node `Marker3D` como filho direto do nó raiz e renomeie-o para `PlayerStart`.
2. Posicione o `PlayerStart` no ponto onde o Player deve começar a explorar o nível.
3. Com o `PlayerStart` selecionado, abra a subaba **Node > Groups** e adicione-o ao grupo `player_start`.
4. Selecione o Node do Player na cena (ou a raiz da Scene `Player`, conforme a organização do grupo) e adicione-o ao grupo `player`.
5. Abra `game_manager.torch` no editor do Orchestrator e crie uma **função pública** chamada `spawn_player` (sem parâmetros, sem retorno). A lógica do grafo deve reproduzir esta implementação de referência em GDScript:


```
func spawn_player() -> void:
    var player := get_tree().get_first_node_in_group("player")
    var inicio :=
get_tree().get_first_node_in_group("player_start")
    if player and inicio:
        player.global_position = inicio.global_position
```

No grafo: dois nós `get_tree().get_first_node_in_group(...)` (um com `"player"`, outro com `"player_start"`) → um nó **Branch** que só segue se ambos forem válidos → um nó **Set** de `global_position` do `player` recebendo `global_position` do `inicio`.

6. No nó raiz de `level_exploration.tscn` (via sua Orchestration ou script), no evento `_ready`, chame a função do Autoload:

```
func _ready() -> void:
    GameManager.spawn_player()
```

No Orchestrator: evento `_ready` → nó **Call Function** `spawn_player` com o alvo vindo de um nó **Get Autoload** (`GameManager`).

7. Rode `level_exploration.tscn` (F6) e confirme que o Player aparece na posição do `PlayerStart`.
8. Pare a execução, arraste o `PlayerStart` para outro lugar do nível, rode de novo e confirme que o Player agora nasce na nova posição — sem nenhuma alteração no grafo.

Resultado esperado

Ao carregar o nível, o Player é sempre reposicionado na coordenada do `PlayerStart` pelo `GameManager`. Mover o marcador no editor muda o ponto de partida; a Orchestration `game_manager.torch` não contém nenhuma coordenada.

Verificando

1. Mova o `PlayerStart` e confirme que o Player segue a nova posição ao rodar.
2. Desligue temporariamente a chamada `GameManager.spawn_player()` no `_ready()` do nível e confirme que o Player volta a nascer onde a instância dele está salva na cena — provando que é o `GameManager` que decide o spawn.
3. Confirme que `game_manager.torch` não tem nenhuma variável de posição (`Vector3`, `Transform3D`) — apenas a leitura dos grupos.

Problemas comuns

- Guardar a posição inicial como variável no `GameManager` (uma variável `Vector3` no grafo) em vez de ler o `Marker3D`: a coordenada é dado da cena — o `GameManager` só decide *usar* o `PlayerStart`, não *onde ele fica*.
- Esquecer de adicionar o Player ao grupo `player` ou o marcador ao grupo `player_start`: `get_first_node_in_group()` retorna `null` e o Player não é movido.
- Chamar `spawn_player()` antes de o Player existir na árvore (por exemplo, no evento `_ready` do próprio Autoload): usar o `_ready` do nó raiz do nível, que roda depois de a cena inteira ser instanciada.
- Esquecer de marcar a função `spawn_player` como pública/chamável externamente na Orchestration: nesse caso `GameManager.spawn_player()` não é reconhecido pelos outros nós/scripts.

Boas práticas

- Um único `PlayerStart` por nível nesta fase — múltiplos pontos de entrada só fazem sentido com transições entre níveis, fora do escopo do Vertical Slice.
- Nomear a função `spawn_player()` de forma que a intenção fique clara — ela será reutilizada pelo respawn após morte no Módulo 3.

Comparação com Unity

A Unity não tem um actor `PlayerStart` nativo nem um `ChoosePlayerStart`: o padrão comum é um `GameObject` vazio marcado como "SpawnPoint", encontrado por tag ou referência serializada, e um script gerenciador que move o Player para lá em `Start()` / `OnSceneLoaded`. A estrutura — coordenada no objeto de cena, decisão no gerenciador — é a mesma; muda apenas que Godot e Unity deixam o time montar o padrão, enquanto a Unreal o entrega pronto.

Ao final do encontro

Ao final deste encontro, o projeto do Vertical Slice deve conter:

- O Player, o nível de teste e o build da Semana 3, sem alterações além dos grupos `player` / `player_start` e da chamada de spawn no `_ready()` do nível.
- Uma Orchestration `orchestrations/autoload/game_manager.torch` (classe base `Node`), com a função pública `spawn_player()`.

- O `GameManager` registrado e habilitado na aba Autoload de Project Settings (apontando para `game_manager.torch`), com acesso validado a partir do Player.
- Um `Marker3D PlayerStart` em `level_exploration.tscn` (grupo `player_start`), consultado pelo `GameManager` ao carregar o nível.

Segundo o `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 2), este resultado corresponde ao início do item "GameManager (Autoload)" do roadmap, já com sua primeira responsabilidade concreta (`spawn_player()`). O Encontro 2 desta semana completa a variável de estado própria (desafio) e introduz o `SaveManager`.

Desafio

Cada estudante adiciona ao `GameManager` uma variável de estado de partida própria, não demonstrada neste tutorial — por exemplo, um contador de tentativas, um estado de progresso simples (`progresso_atual: int = 0`) ou uma flag de evento (`porta_principal_aberta: bool = false`). A variável deve ser declarada no painel de variáveis da Orchestration `game_manager.torch` e seu valor deve ser lido com sucesso a partir de um teste temporário no Player (nó `Print / print()`), da mesma forma testada na Parte 2. Justifique brevemente, numa nota no próprio grafo, por que essa variável pertence ao `GameManager` e não a uma Scene específica.

Checklist

- ☐ Pasta `orchestrations/autoload/` criada, seguindo a estrutura do `PROJECT_ARCHITECTURE.md`
- ☐ Orchestration `game_manager.torch` criada (classe base `Node`), com nota de responsabilidade registrada
- ☐ `GameManager` registrado e habilitado em **Project Settings > Autoload**, apontando para `game_manager.torch`
- ☐ Acesso ao `GameManager` testado com sucesso a partir do Player
- ☐ `Marker3D PlayerStart` adicionado a `level_exploration.tscn` e ao grupo `player_start`; Player no grupo `player`
- ☐ Função pública `spawn_player()` implementada no grafo (lendo os grupos, sem coordenada) e chamada no `_ready()` do nível
- ☐ Player confirmadamente reposicionado no `PlayerStart` ao rodar; mover o marcador muda o spawn

□ Variável de estado própria do desafio adicionada ao `GameManager`, com justificativa anotada no grafo

Glossário

- **Autoload/Singleton:** mecanismo nativo do Godot que instancia automaticamente um recurso (script, Orchestration ou cena) na raiz da árvore de cenas ao iniciar o jogo, tornando-o acessível globalmente por nome e sobrevivente a trocas de cena.
- **Orchestration:** grafo de visual scripting do Orchestrator (arquivo `.torch`) — a camada de lógica principal da disciplina. Pode ser registrada diretamente como Autoload, como um `.gd`.
- **Implementação de referência:** trecho de GDScript apresentado no tutorial para descrever, de forma precisa, a lógica que o grafo do Orchestrator deve reproduzir; não é um arquivo a ser digitado.
- **GameManager:** Autoload responsável por centralizar regras de partida e estado compartilhado do Vertical Slice, reunindo o que a Unreal separa em `GameMode` e `GameState`.
- **Singleton:** padrão de projeto que garante que uma classe possua uma única instância acessível globalmente; no Godot, implementado nativamente via Autoload.
- **Função pública (Orchestrator):** função de uma Orchestration marcada como chamável externamente, permitindo que outros scripts/grafos a invoquem por `NomeDoAutoload.nome_da_funcao()`.
- **Get Autoload (Orchestrator):** nó que devolve a referência de um Autoload registrado, para chamar suas funções e ler suas variáveis a partir de outro grafo.
- **PlayerStart:** `Marker3D` colocado no nível (grupo `player_start`) que guarda a coordenada onde o Player nasce ao carregar o nível — equivalente ao actor `PlayerStart` da Unreal.
- **spawn_player()**: função do `GameManager` que reposiciona o Player no ponto de spawn ao carregar o nível (e, no Módulo 3, no respawn após morte) — o equivalente a `GameMode.ChoosePlayerStart`.
- **Grupo (Group):** rótulo atribuído a Nodes no Godot que permite localizá-los por nome (`(get_tree().get_first_node_in_group(...))`) sem referência direta nem caminho fixo.

Referências

- Godot Documentation — Singletons (Autoload): https://docs.godotengine.org/en/stable/tutorials/scripting/singletons_autoload.html
- Godot Documentation — GDScript: <https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/index.html>
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Orchestrator — Autoloads: <https://docs.cratercrash.space/orchestrator/nodes/autoloads/>
- Unity Manual (consulta comparativa) — DontDestroyOnLoad: <https://docs.unity3d.com/ScriptReference/Object.DontDestroyOnLoad.html>
- GDQuest: <https://www.gdquest.com/>